



5-Phase SRM Drive · Tuk-Tuk Feasibility Study

Student Lab Guide · Fast Read · OpenModelica + EmbedSim

Motor	5-Phase 10/8 SRM — no rare-earth magnets
Vehicle	GVW 637 kg target 45 km/h
Power	4.5 kW peak 48 V DC bus (touch-safe LiFePO ₄)
Sensor	Resolver → RDC → angle & speed feedback
Plant	OpenModelica model exported as FMU 2.0
Simulation	EmbedSim Python framework (FmuBlock + VectorBlocks)
Controllers	PI (baseline) → Sliding Mode → Differential Flatness
Code output	MISRA C:2012 generated by EmbedSim CodeGen
Process	V-Model development lifecycle

Abstract

Build and simulate a complete SRM motor drive for a Sri Lankan three-wheeler. Labs 0–6 take you from a hand-calculation feasibility study through OpenModelica physical modelling, FMU export, EmbedSim closed-loop simulation, PI and sliding-mode controllers, and V-Model traceability to embedded C code generation on AURIX TC3xx.

Lab 0 FEASIBILITY STUDY — DOES AN SRM WORK FOR A TUK-TUK?

Before any design: derive minimum motor power and torque from real vehicle data. This is the System Requirements level of the V-Model. Every number is traceable to a physical constraint.

Three resistive forces to calculate:

- $F_{roll} = C_{rr} \times m \times g$ (rolling resistance — constant with speed)
- $F_{aero} = 0.5 \times \rho \times C_{nA} \times v^2$ (aerodynamic drag — grows as v^2)
- $F_{grade} = m \times g \times \sin(\arctan(i))$ (hill climb — grade $i = 0.15$ for 15%)
- $F_{accel} = m \times a$ (Newton's second law for the acceleration phase)

Four operating conditions to verify (worst-case for each):

- Cruise at 45 km/h flat: $F_{roll} + F_{aero}(45) \rightarrow$ motor shaft torque $T_m = T_{wheel} \div (G \times \eta)$
- Hill climb at 20 km/h (15% grade): $F_{roll} + F_{grade}$
- Acceleration 0–40 km/h flat: $F_{roll} + F_{accel}$
- Worst case — acceleration on hill: $F_{roll} + F_{grade} + F_{accel}$

Motor speed at 45 km/h: $n_{\text{motor}} = v_{\text{max}} \times G \div (2\pi \times r_w) \times 60 \approx 2400$ RPM. Feasibility passes if cruise and hill-climb power each stay under 4.5 kW individually. Output: a requirements table that becomes the acceptance criteria for Labs 1–5.

BG WHAT IS A SWITCHED RELUCTANCE MOTOR?

No magnets, no rotor windings — just shaped steel. Torque is produced because magnetic flux always takes the path of least reluctance. When a stator coil fires, the nearest rotor tooth is pulled toward alignment. Five phases, always at least one producing torque — no dead spots.

Phase voltage equation: $v = R \cdot i + L(\theta) \cdot (di/dt) + i \cdot (dL/d\theta) \cdot \omega$

Phase torque equation: $T = 0.5 \cdot i^2 \cdot dL/d\theta$ (positive when inductance is rising)

Key insight: torque $\propto i^2$. Double the current → four times the torque, but also four times the copper loss. The controller must switch phases at the right angles to keep torque positive and avoid demagnetising during the falling-inductance region.

Lab 1 BUILD THE OPENMODELICA PHASE MODEL

Build a single SRM phase model in OMEdit. Key parameters: $R = 0.045 \Omega$, $L_{\text{unaligned}} = 0.9 \text{ mH}$, $L_{\text{aligned}} = 4.5 \text{ mH}$, $N_r = 8$ rotor poles.

- Task 1.1 — Create SRM_Phase model; declare R , L_u , L_a , N_r as parameters with SI units
- Task 1.2 — Add inductanceProfile() function: piecewise-linear triangular $L(\theta)$ with 8 peaks per revolution (one per rotor pole). Plot L vs θ : should show 8 identical triangles between 0.9 mH and 4.5 mH
- Task 1.3 — Write the phase voltage ODE: $v = R \cdot i + L \cdot \text{der}(i) + i \cdot (dL/d\theta) \cdot \omega$. Use finite-difference for $dL/d\theta$. Verify: step 48 V at $\theta = 0.3 \text{ rad}$, $\omega = 0 \rightarrow$ current rises from 0 A

Lab 2 ASSEMBLE THE FULL MOTOR AND EXPORT AS FMU

Connect 5 phase models to a rotor mechanics block and export the complete plant as FMU 2.0 for EmbedSim.

- Task 2.1 — Build SRM_Rotor: $J \cdot d\omega/dt = T_{\text{net}} - B \cdot \omega$, $d\theta/dt = \omega$. Parameters: $J = 0.010 \text{ kg} \cdot \text{m}^2$, $B = 0.002 \text{ N} \cdot \text{m} \cdot \text{s/rad}$
- Task 2.2 — Assemble SRM_Plant_10_8: 5 phase instances + 1 rotor. Phase angle offsets: 0, $2\pi/5$, $4\pi/5$, $6\pi/5$, $8\pi/5$. Sum all phase torques – T_{load} to drive rotor
- Task 2.3 — Export FMU 2.0 Co-Simulation from OMEdit. Unzip and verify v_{in} , T_{load} , ω_{out} , θ_{out} , i_{out} appear in modelDescription.xml

Lab 3 OPEN-LOOP TEST IN EMBEDSIM

Run the plant open-loop with a fixed voltage pattern before adding a controller. FmuBlock wraps the OpenModelica FMU so it behaves like any other EmbedSim block.

- Task 3.1 — Set up EmbedSim project: FmuBlock('SRM_Plant_10_8.fmu', dt=50e-6), SimulationEngine, SimulationLogger
- Task 3.2 — Apply fixed-angle commutation: excitation window 0–0.26 rad, freewheeling 0.26–0.38 rad, demagnetise –0 V otherwise. $V_{dc} = 48$ V, $T_{load} = 5$ N·m. Expected: 150–200 rad/s (1400–1900 RPM) within 1 s, current pulses 30–50 A
- Task 3.3 — Plot ω_{out} , all 5 phase currents. Compute torque ripple = $(T_{max} - T_{min}) / T_{mean} \times 100\%$. Record — compare to closed-loop result in Lab 5

Lab 4 ADD A PI SPEED CONTROLLER

PI control law: $u(t) = K_p \cdot e(t) + K_i \cdot \int e(t) dt$, where $e = \omega_{ref} - \omega_{meas}$. K_p gives immediate response; K_i removes steady-state error. Anti-windup freezes the integrator when the output hits the torque clamp — prevents overshoot on large steps.

- Task 4.1 — Implement SRM_PI_SpeedController(K_p , K_i , $T_{max}=25$ N·m). Clamp T_{ref} to $[0, T_{max}]$. Anti-windup: undo last integration step when output is clamped
- Task 4.2 — Wire closed loop: plant.omega_out → pi.omega_meas, pi.T_ref → commutation → plant.v_in. Set $\omega_{ref} = 2400$ RPM, $T_{load} = 8$ N·m
- Task 4.3 — Tune gains. Target: settling < 1.5 s, overshoot < 10%, zero steady-state error. With $K_p = 0.05$, $K_i = 0.5$: expect settling ≈ 1.2 s, overshoot $\approx 8\%$

Lab 5 SLIDING MODE CONTROLLER AND COMPARISON

Sliding surface: $s = e\omega + c \cdot \int e\omega dt$. The controller switches to push the system onto $s = 0$ and keep it there. A boundary layer (sat function) smooths the switching to reduce chattering. More robust to load disturbances than PI.

- Task 5.1 — Implement SRM_SMC_SpeedController(J , B , $c=20$, $K_{sw}=5$, $\phi=0.05$). T_{eq} = feedforward term ($J \cdot \omega_{ref_dot} + B \cdot \omega + T_{load_est}$). $T_{sw} = -K_{sw} \cdot \text{sat}(s/\phi)$
- Task 5.2 — Run 3 scenarios with both PI and SMC:
 - Scenario A** — speed step 0 → 2400 RPM, constant load 8 N·m. Measure: rise time, overshoot, settling time
 - Scenario B** — load step 8 → 18 N·m at $t = 2$ s (tuk-tuk hits a hill). Measure: speed dip (RPM), recovery time
 - Scenario C** — phase 3 dead ($v_{in}[2] = 0$). Measure: achieved steady-state speed, torque ripple increase
- Task 5.3 — Reflection: Why does SRM produce torque ripple? What happens with K_{sw} too small / too large? Effect of missing anti-windup? Resolver resolution at 2400 RPM?

Typical results: SMC gives 30–50% faster disturbance recovery in Scenario B, maintains speed within 50 RPM in Scenario C (PI may drop 150–200 RPM). PI generally has less overshoot in Scenario A with careful tuning.

Lab 6 THE V-MODEL — WHERE DOES YOUR SIMULATION FIT?

No simulation in this lab. Map every file you created onto the V-Model — the engineering process standard used in automotive development (ISO 26262, AUTOSAR).

- System level: Lab 0 power worksheet = left side (requirement derivation). Physical tuk-tuk road test = right side (acceptance test)
- Module level: SRM_Phase.mo, SRM_Plant_10_8.mo = left (plant model). Open-loop commutation test = right (first plant verification)
- Component level: PI/SMC gain tuning table = right (controller parameter verification)
- Code level: embedsim_step.c from CodeGen = left (implementation). MISRA checker run = right (code verification)

Task 6.1: Trace two Scenario A/B/C results back to a Lab 0 system requirement. State pass/fail. If fail: identify which block or gain to change.

Task 6.2: List 5 missing verification activities before hardware installation (thermal testing, EMC, resolver calibration, fail-safe behaviour, road test on Sri Lankan roads).

Task 6.3: Why is the FMU plant excluded from CodeGen? What replaces it on AURIX TC3xx? What happens if the C code takes 60 μ s when the ISR period is 50 μ s?

REF QUICK REFERENCE — KEY NUMBERS AND CODE PATTERNS

Key numbers to memorise:

- DC bus: 48 V (touch-safe LiFePO_4) | Peak phase current: 80 A | Continuous: 45 A RMS
- $L_{\text{aligned}} = 4.5 \text{ mH}$ | $L_{\text{unaligned}} = 0.9 \text{ mH}$ | Rated speed = 2400 RPM = 251 rad/s
- Rotor pole pitch = 45° (0.785 rad) | Phase offset = 72° (1.257 rad) | FMU step = 50 μ s
- Vehicle load at 45 km/h $\approx 12 \text{ N}\cdot\text{m}$ motor shaft torque (GVW 637 kg, flat road)

EmbedSim patterns:

- Load FMU: plant = FmuBlock('motor.fmu', dt=DT)
- Connect blocks: topo.connect(blockA, 'out_name', blockB, 'in_name')
- Run: engine = SimulationEngine(topo, logger, dt, t_end); engine.run()
- Clip value: $x = \text{np.clip}(x, \text{lo}, \text{hi})$ | SMC sat: s/ϕ if $\text{abs}(s) < \phi$ else $\text{np.sign}(s)$

OpenModelica patterns:

- Parameter: parameter Real Kp = 1.0 "Description [unit]";
- ODE: $J * \text{der}(\omega) = T_{\text{net}}$; // $J \cdot d\omega/dt = T$
- Export: FMI → Export FMU → Co-Simulation 2.0 | Validate: unzip → modelDescription.xml